SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

CO1-AT1-Analytical Problem Solving

Register Number	**192511009**
Name	**Avishek N**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks: 50**

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

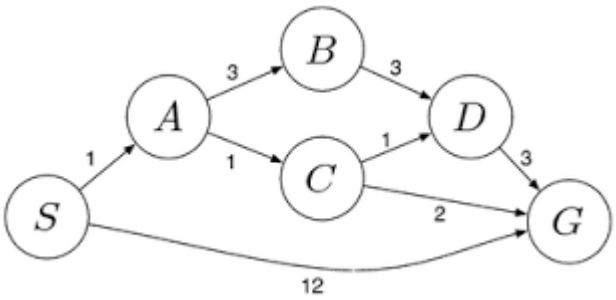
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost

path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Code of Conduct:

I Avishek N(192511009). certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Avishek N

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

Question 1

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Answer:

Each state is represented as a pair (x, y), where x = amount of water in the 4-gallon jug and y = amount of water in the 3-gallon jug. The goal test is x = 2. The available operators (production rules) are:

- Fill the 4-gallon jug completely from the pump.
- Fill the 3-gallon jug completely from the pump.
- Empty the 4-gallon jug onto the ground.
- Empty the 3-gallon jug onto the ground.
- Pour water from the 4-gallon jug into the 3-gallon jug until the 4-gallon jug is empty or the 3-gallon jug is full.
- Pour water from the 3-gallon jug into the 4-gallon jug until the 3-gallon jug is empty or the 4-gallon jug is full.

Using Breadth-First Search over this state space gives the shortest sequence of moves to the goal.

Step	State (4-gal, 3-gal)	Action taken
0	(0, 0)	Initial state
1	(4, 0)	Fill the 4-gallon jug
2	(1, 3)	Pour from 4-gallon jug into 3-gallon jug
3	(1, 0)	Empty the 3-gallon jug
4	(0, 1)	Pour from 4-gallon jug into 3-gallon jug
5	(4, 1)	Fill the 4-gallon jug
6	(2, 3)	Pour from 4-gallon jug into 3-gallon jug — GOAL

Result: Exactly 2 gallons of water are obtained in the 4-gallon jug in 6 moves.

Algorithm
Breadth First Search
Step 1
Start with the initial state.
Step 2
Generate all possible legal moves.
Step 3
Store unexplored states in a queue.
Step 4
Repeat until the goal state is found.

Python Code:

File Edit Format Run Options Window Help

```
from collections import deque
def water_jug():
    visited = set()
    queue = deque()
    queue.append((0,0,[]))
    while queue:
        x,y,path = queue.popleft()
        if (x,y) in visited:
            continue
        visited.add((x,y))
        path = path + [(x,y)]
        if x == 2:
            return path
        next_states = [
            (4,y),
            (x,3),
            (0,y),
            (x,0),
            (x-min(x,3-y), y+min(x,3-y)),
            (x+min(y,4-x), y-min(y,4-x))
        ]
        for state in next_states:
            if state not in visited:
                queue.append((state[0],state[1],path))

solution = water_jug()
print("Solution Path")
for state in solution:
    print(state)
```

Output:

File Edit Shell Debug Options Window Help

```
Python 3.4.3 (v3.4.3:9b73f1c3e601, Feb 24 2015, 22:44:40) [MSC v.1600 64 bit (AM
D64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ================================ RESTART ================================
>>>
Solution Path
(0, 0)
(4, 0)
(1, 3)
(1, 0)
(0, 1)
(4, 1)
(2, 3)
>>> ================================ RESTART ================================
```

Flowchart:

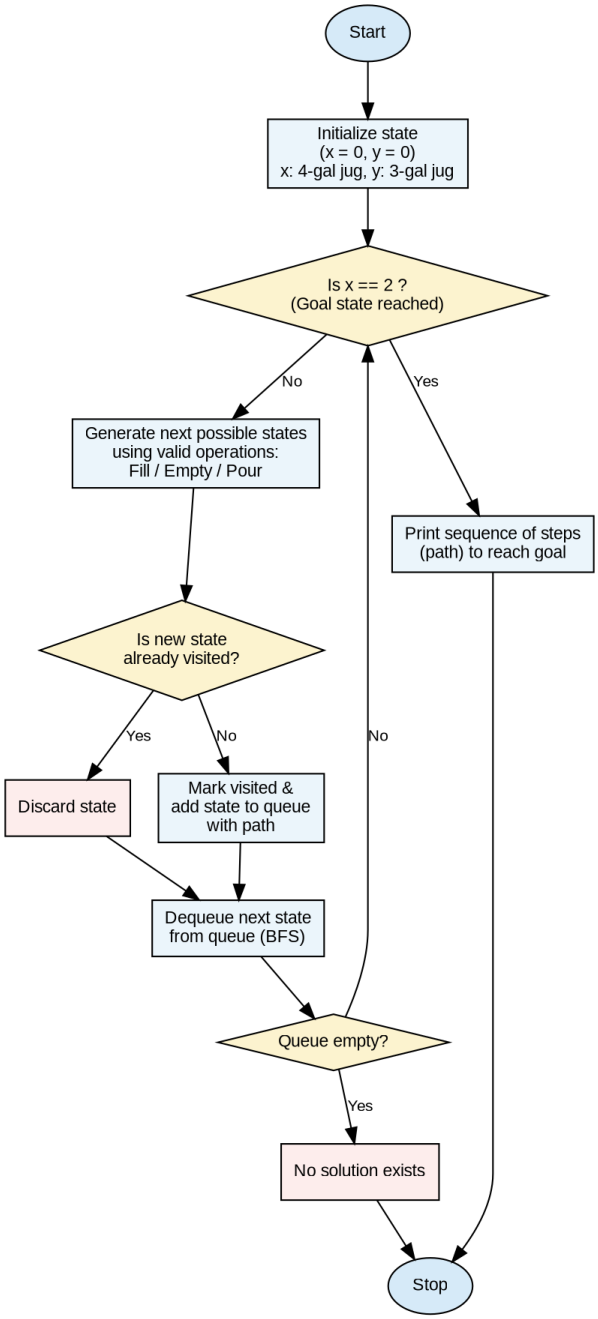


Fig 1.1 — Flowchart of the BFS-based Water Jug solution

Advantages:

- Guarantees the shortest solution using BFS.
- Easy to understand and implement.
- Demonstrates state-space search effectively.
- Widely used to teach AI search algorithms.

The Water Jug Problem illustrates how Artificial Intelligence solves real-world problems using state-space search techniques. By representing each configuration as a state and exploring all valid operations systematically, the Breadth First Search algorithm successfully reaches the goal state **(2,0)** with the minimum number of steps.

Question 2

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   vi. What are the percept's for this agent?
   vii. Characterize the operating environment.
   viii. What are the actions the agent can take?
   ix. How can one evaluate the performance of the agent?
   x. What sort of agent architecture do you think is most suitable for this agent? Why?

Answer:

Introduction

A **Mars Rover** is an autonomous robotic vehicle designed to explore the surface of Mars. It performs scientific experiments, captures images, analyzes rocks and soil, collects samples, and sends information back to Earth. Since communication between Earth and Mars has a long delay, the rover must make many decisions independently using Artificial Intelligence.

PEAS Description

Component	Description
Performance Measure	Accurate exploration, safe navigation, successful sample collection, energy efficiency, reliable communication
Environment	Mars surface, rocks, mountains, dust storms, craters, extreme temperatures
Actuators	Wheels, robotic arm, drill, camera, antenna, sample collector
Sensors	Cameras, GPS-like localization system, temperature sensors, infrared sensors, spectrometers, obstacle sensors

(a) Percepts of the agent:

- Camera / visual images of the Martian terrain and rock formations
- Spectrometer and sensor readings of soil / rock composition
- GPS-like positional and orientation data (from onboard navigation sensors)
- Obstacle distance readings from LIDAR / ultrasonic sensors
- Battery / power level and temperature readings

(b) Characterization of the operating environment:

- Partially observable — the rover can only sense its immediate surroundings, not the whole planet surface
- Stochastic — terrain conditions (dust storms, slippage) introduce unpredictability
- Sequential — past actions/movements affect future states (battery used, position changed)

- Dynamic — Martian weather and lighting change over time independent of the rover
- Continuous — position, sensor values and time are continuous quantities
- Single agent — (assuming one rover operating without cooperating agents)

(c) Actions the agent can take:

- Move forward / backward, turn left / right
- Capture images of terrain and samples
- Drill / collect soil or rock samples
- Analyse collected samples using onboard instruments
- Transmit data back to the base station on Earth
- Recharge / enter low-power sleep mode

(d) Performance evaluation of the agent:

- Number and scientific value of samples successfully analysed
- Distance covered safely without collisions or getting stuck
- Efficient use of battery power and time
- Accuracy and completeness of data transmitted to Earth
- Ability to avoid/recover from hazards (e.g., steep slopes, sand traps)

(e) Most suitable agent architecture:

A Model-based, goal-based (and ideally learning) agent architecture is most suitable. Because the environment is partially observable, the rover must maintain an internal model/map of Mars built from past percepts to make sense of what it currently senses. Because it must decide where to drill or which sample to collect, it needs explicit goals (e.g., "reach coordinates X and collect a sample") rather than simple reflexes. Finally, a learning component allows the rover to refine its terrain model and improve obstacle-avoidance and sample-selection strategies over time, which is valuable given the extremely limited opportunity for direct human intervention (communication delay with Earth).

Python Code:

File Edit Format Run Options Window Help

```
class MarsRover:

    def move(self):
        print("Moving Forward")

    def capture_image(self):
        print("Capturing Image")

    def analyze_sample(self):
        print("Analyzing Rock Sample")

    def send_data(self):
        print("Sending Data to Earth")

rover = MarsRover()

rover.move()
rover.capture_image()
rover.analyze_sample()
rover.send_data()
```

Output:

```
>>> ================================
>>>
Moving Forward
Capturing Image
Analyzing Rock Sample
Sending Data to Earth
>>> ================================
```

Advantages

- Performs autonomous exploration.
- Reduces human intervention.
- Collects accurate scientific data.
- Works in hazardous environments.
- Learns from previous experiences.
- Saves mission time.

Applications

- Space exploration
- Planetary research
- Geological surveys
- Autonomous robotics
- Military reconnaissance
- Disaster response

The Mars Rover is an excellent example of an intelligent agent in Artificial Intelligence. By using sensors, actuators, and a learning agent architecture, it can navigate autonomously, analyze Martian samples, avoid obstacles, and send valuable scientific information back to Earth. Its ability to learn and adapt makes it highly effective for long-duration space exploration missions.

Question 3

3.Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Answer:

Problem formulation (incremental / complete-state formulation):

- States: Any arrangement of 0 to 8 queens on the board, at most one queen per column.
- Initial state: An empty chessboard (no queens placed).
- Actions: Add a queen to any empty square in the leftmost empty column such that it is not attacked by any previously placed queen.
- Transition model: Returns the board with the new queen added at the specified square.
- Goal test: 8 queens are placed on the board and no two queens attack each other (no shared row, column, or diagonal).
- Path cost: Not relevant here (all actions/steps are of equal, unit cost); we only care about reaching a valid goal state.

Search strategy: Because the branching factor grows very large with a naive formulation, we use Backtracking Search (a form of depth-first search) — queens are placed column by column; whenever a partial placement is found unsafe, the algorithm backtracks and tries the next possible row in that column, instead of generating the full state space blindly

Time Complexity

O(N!)

For 8 queens,

O(8!) = 40320

Backtracking explores possible arrangements but prunes invalid ones.

Space Complexity

O(N)

Because recursion stores one queen placement per row.

Algorithm

1. **Start from the first row.**
2. **Place a queen in a safe column.**
3. **Move to the next row.**
4. **Check whether the queen attacks another queen.**
5. **If safe, continue.**
6. **Otherwise, remove the queen (Backtrack).**
7. **Try the next column.**
8. **Repeat until all 8 queens are placed safely.**

Python Code:

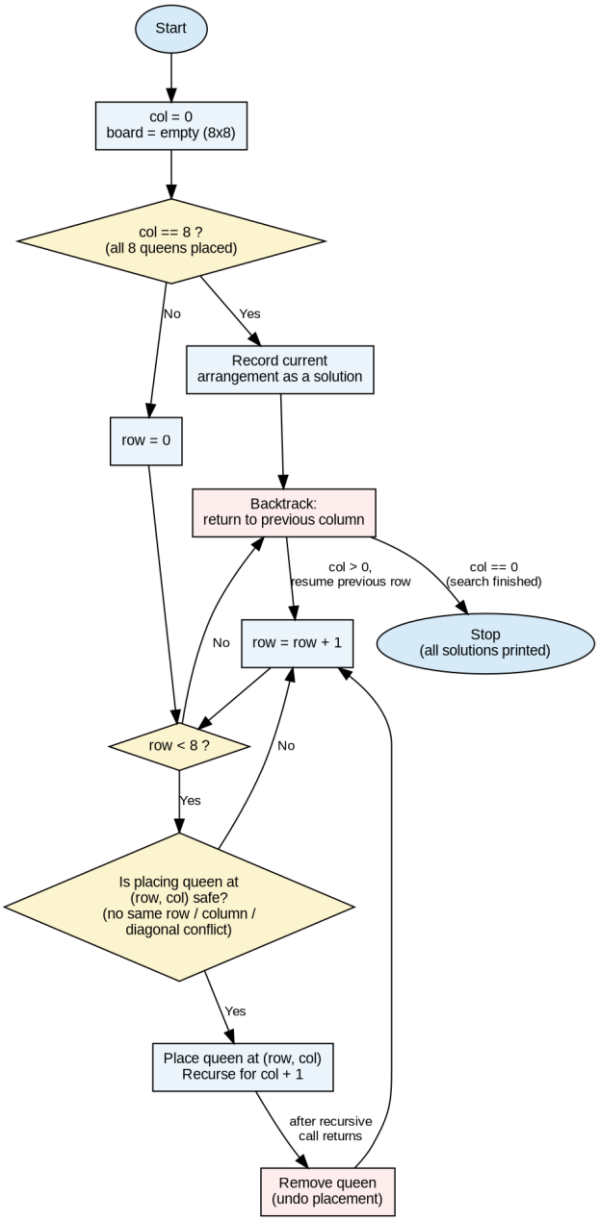
File Edit Format Run Options Window Help

```
N = 8
board = [[0] * N for _ in range(N)]
def isSafe(row, col):
    for i in range(col):
        if board[row][i]:
            return False
    i = row
    j = col
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1
    i = row
    j = col
    while i < N and j >= 0:
        if board[i][j]:
            return False
        i += 1
        j -= 1
    return True
def solve(col):
    if col >= N:
        return True
    for i in range(N):
        if isSafe(i, col):
            board[i][col] = 1
            if solve(col + 1):
                return True
            board[i][col] = 0
    return False
solve(0)
for row in board:
    print(row)
```

Output:

```
>>> ================================ RESTART
>>>
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
>>> ================================ RESTART
```

Flowchart:



The **8-Queens Problem** is a classic Constraint Satisfaction Problem in Artificial Intelligence. By using the **Backtracking Search Algorithm**, the system systematically explores possible queen placements, eliminates invalid configurations, and backtracks whenever a conflict occurs. This approach guarantees a correct solution while avoiding unnecessary computations, making it one of the most effective techniques for solving chessboard placement problems.

Question 4

4.A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem

Answer:

The OLA cab booking scenario is best modelled as a Goal-Based Problem-Solving Agent. The customer's pickup location is the initial state and the destination is the goal state; the agent must search among several possible actions (cab types) and select the sequence of actions that satisfies the customer's goal at the best cost/comfort trade-off — a hallmark of goal-based, problem-solving agent behaviour rather than a simple reflex agent.

Problem formulation:

- Initial state: Customer's current pickup location.
- Goal state: Customer's chosen destination location.
- Actions: Choose a cab type (Mini, Micro, Sedan, Shared, Prime, etc.) and travel along the route to the destination.
- Path cost: A function of fare, distance, estimated time of arrival, and customer comfort preference.
- Goal test: Customer has been dropped at the destination location.

Type of Intelligent Agent Used
The most suitable agent is a **Goal-Based Problem Solving Agent**.

Why Goal-Based Agent?
A goal-based agent chooses actions that help achieve a specific goal.

Goal:
Book the best available cab and transport the customer safely to the destination.
The agent evaluates different alternatives before selecting the best one.

Working of the OLA Agent

1. Customer opens the OLA application.
2. Customer enters pickup location.
3. Customer enters destination.
4. AI checks nearby drivers.
5. Calculates estimated fare.
6. Displays available cab types.
7. Customer selects preferred cab.
8. Booking request is sent to the nearest driver.
9. Driver accepts the request.
10. Trip starts.
11. Customer reaches destination.
12. Payment is completed.

Python Code:

```
File  Edit  Format  Run  Options  Window  Help
pickup = input("Enter Pickup Location: ")
destination = input("Enter Destination: ")

cab = input("Choose Cab (Mini/Micro/Sedan/Prime): ")

print("\nSearching nearby drivers...")

print("Driver Found!")

print("Cab Type:", cab)

print("Trip Started...")

print("Travelling from", pickup, "to", destination)

print("Destination Reached Successfully!")

print("Payment Completed.")
```

Output:

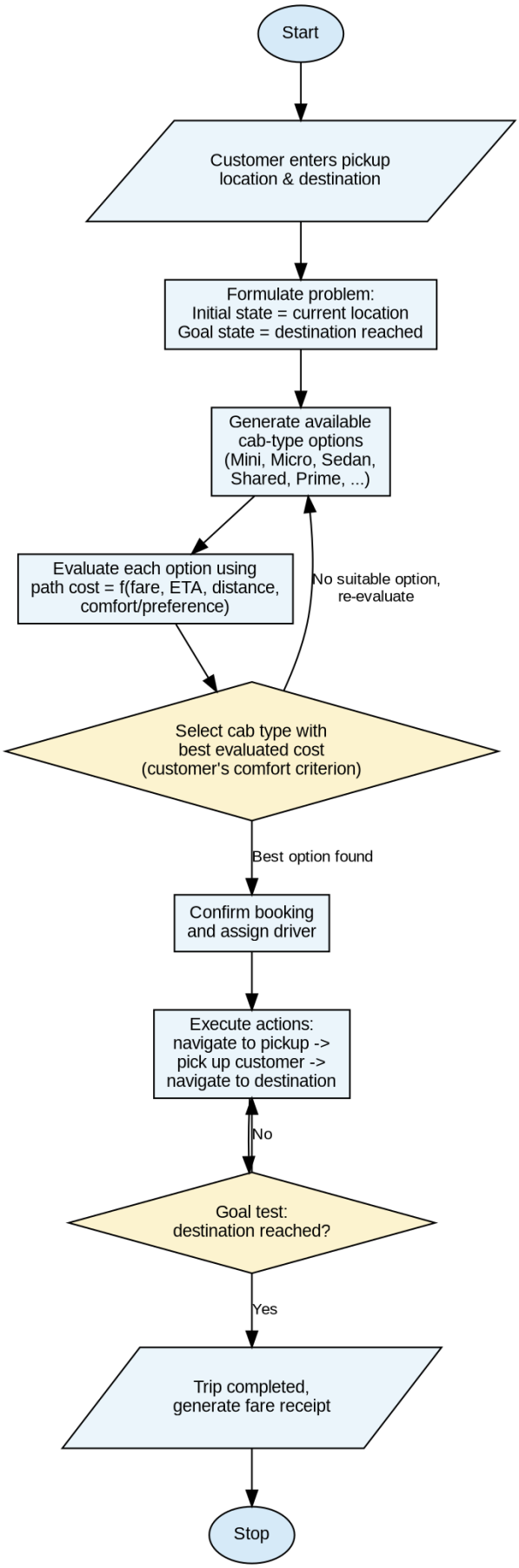
```
>>>
Enter Pickup Location: chennai
Enter Destination: ayapakkam
Choose Cab (Mini/Micro/Sedan/Prime): sedan

Searching nearby drivers...
Driver Found!
Cab Type: sedan
Trip Started...
Travelling from chennai to ayapakkam
Destination Reached Successfully!
Payment Completed.
>>>
```

Algorithm

Goal-Based Search Algorithm

Step 1: Start the application.
Step 2: Enter pickup location.
Step 3: Enter destination.
Step 4: Search nearby drivers.
Step 5: Display available cab types.
Step 6: Select the most suitable cab.
Step 7: Send booking request.
Step 8: If accepted, begin journey.
Step 9: Reach destination.
Step 10: Complete payment.

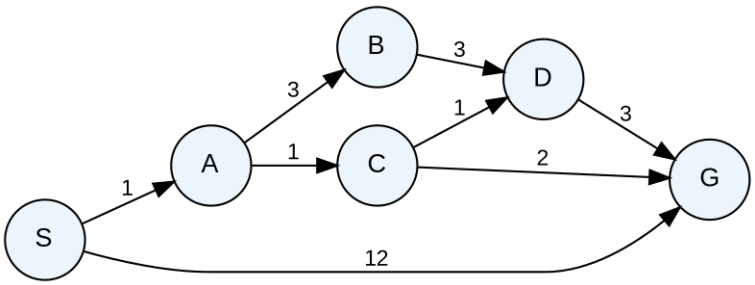
Flowchart:


Start
Customer enters pickup location & destination
Formulate problem: Initial state = current location Goal state = destination reached
Generate available cab-type options (Mini, Micro, Sedan, Shared, Prime, ...)
Evaluate each option using path cost = f(fare, ETA, distance, comfort/preference)
No suitable option, re-evaluate
Select cab type with best evaluated cost (customer's comfort criterion)
Best option found
Confirm booking and assign driver
Execute actions: navigate to pickup -> pick up customer -> navigate to destination
No
Goal test: destination reached?
Yes
Trip completed, generate fare receipt
Stop

Question 5

5.A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Answer:

Order	Node expanded (cost)	Frontier update
1	S (0)	Expand S → A(1), G(12)
2	A (1)	Expand A → B(1+3=4), C(1+1=2)
3	C (2)	Expand C → D(2+1=3), G(2+2=4) [better than G(12), updated]
4	D (3)	Expand D → G(3+3=6) [worse than existing G(4), discarded]
5	G (4)	Goal reached — Path: S → A → C → G, Total cost = 4

Least-cost path: S → A → C → G with a total cost of 4 (compare with S → A → C → D → G = 6, S → A → B → D → G = 10, and the direct edge S → G = 12).

Working of Uniform Cost Search

1. Start from the source node.
2. Insert the source into the priority queue.
3. Expand the node with the **lowest path cost**.
4. Add neighboring nodes with updated costs.
5. Continue until the destination is reached.
6. The first time the goal node is removed from the priority queue, the optimal path is found.

Time Complexity

- **Time Complexity:** O((V + E) log V)
- **Space Complexity:** O(V)

Where:

- **V** = Number of vertices (warehouses)
- **E** = Number of edges (routes)

Python Code:

```
File  Edit  Format  Run  Options  Window  Help
from queue import PriorityQueue
graph = {
    'S': [('A',2), ('B',5)],
    'A': [('C',4), ('D',7)],
    'B': [('D',6), ('E',3)],
    'C': [('G',5)],
    'D': [('G',2)],
    'E': [('G',1)],
    'G': []
}
pq = PriorityQueue()
pq.put((0,'S'))
visited = set()
while not pq.empty():
    cost,node = pq.get()
    if node in visited:
        continue
    visited.add(node)
    print(node,"Cost =",cost)
    if node == 'G':
        print("Goal Reached")
        break
    for next_node,next_cost in graph[node]:
        pq.put((cost+next_cost,next_node))
```

Output:

```
>>> ================================ RESTART
>>>
S Cost = 0
A Cost = 2
B Cost = 5
C Cost = 6
E Cost = 8
D Cost = 9
G Cost = 9
Goal Reached
>>>
```

Advantages

- Finds the optimal (least-cost) path.
- Guarantees the best solution when all costs are non-negative.
- Suitable for weighted graphs.
- Used in many real-world navigation and planning problems.
- More efficient than exhaustive search methods.

Applications

- GPS Navigation Systems
- Google Maps Route Planning
- Logistics and Supply Chain Management
- Robot Path Planning
- Network Routing
- Airline Route Optimization
- Delivery and Courier Services

Conclusion

Uniform Cost Search (UCS) is an optimal uninformed search algorithm that expands the node with the lowest cumulative path cost first. In logistics and delivery systems, UCS helps identify the most cost-effective route, reducing fuel consumption, travel time, and operational expenses. Its ability to guarantee the least-cost solution makes it one of the most widely used search algorithms in Artificial Intelligence.